

Nama : Dhani Arizal Fauzan Adzima

NIM : 245150401111024

Kelas : PAM - F

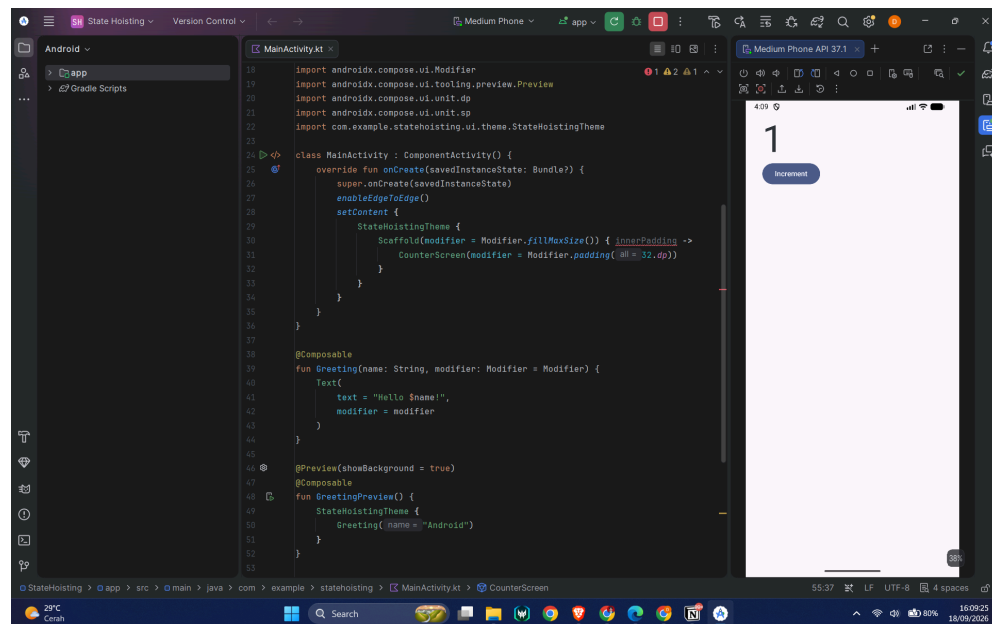
A. State Hoisting

1. Pertanyaan:

- Apa yang terjadi setiap kali tombol "Increment" diklik?
- Apa yang menyebabkan hal tersebut terjadi?
- Variabel *state* apa yang berperan pada Composable **Text** yang menampilkan perubahan angka pada **Text** tersebut?
- Variabel *state* apa yang berperan untuk menampilkan label teks pada Composable **Button** yang bertuliskan "Increment" tersebut?

Jawaban:

- Nilai angka yang ditampilkan pada teks layar bertambah 1 setiap kali tombol ditekan (0, 1, 2, dst.).



- Terjadi proses *recomposition*. Aksi klik pada tombol memicu eksekusi kode `number++` di lambda `onClick` yang memperbarui nilai state `number`, sehingga antarmuka composable digambar ulang dengan nilai yang baru.
- Variabel mutable state `number`, yang dideklarasikan dengan `var number by remember { mutableStateOf(0) }`.

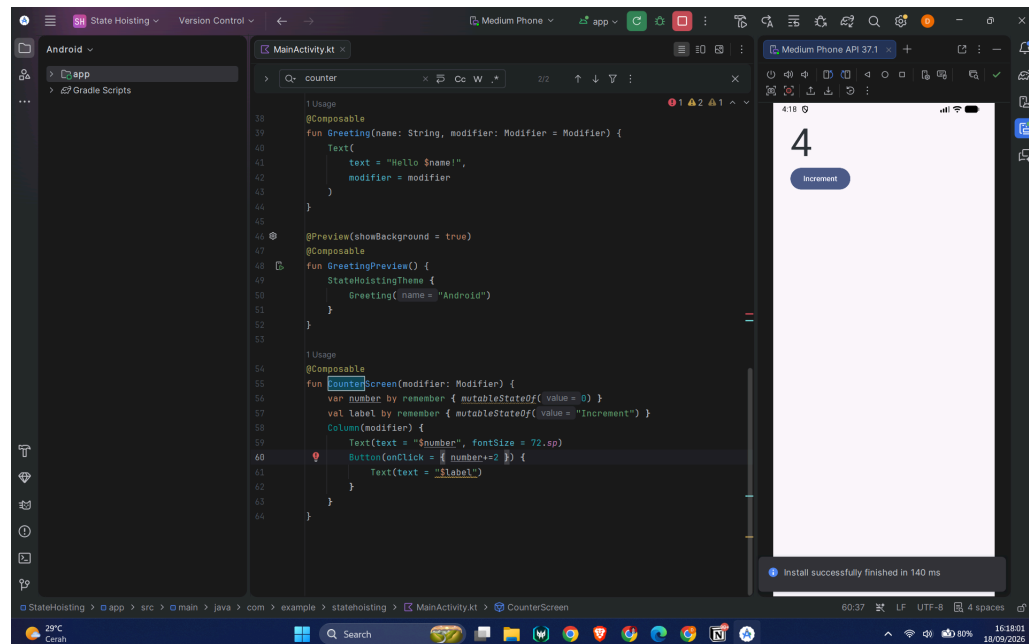
- d. Variabel state `label`, yang dideklarasikan dengan `val label by remember { mutableStateOf("Increment") }`.

2. Pertanyaan:

- a. Perubahan apa yang perlu dilakukan dan dilakukan di mana agar setiap kali tombol "Increment" diklik nilainya bertambah 2 serta label tombol berubah menjadi "Increment 2"?
- b. Bukti pengujian antarmuka sebelum dan sesudah tombol diklik.

Jawaban:

- a. Perubahan dilakukan di dalam body fungsi Composable `CounterScreen` pada file `MainActivity.kt`:
- Mengubah inisialisasi teks label: `val label by remember { mutableStateOf("Increment 2") }`.
 - Mengubah logika penambahan nilai pada event tombol: `Button(onClick = { number += 2 })`.
- b. Dokumentasi

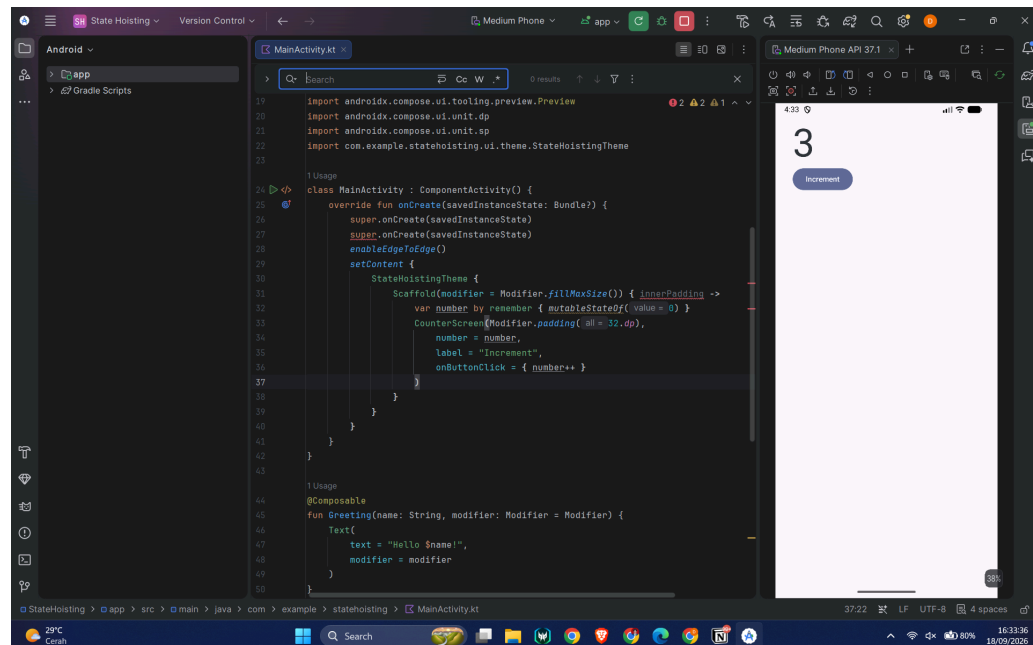


3. Pertanyaan:

- a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap sebelumnya?
- b. Di bagian kode program manakah saat ini proses *increment* dilakukan?
- c. Di bagian kode program manakah saat ini *state* Composable `Text` dan Composable `Button` "Increment" dideklarasikan?

Jawaban:

- Ya, aplikasi berjalan normal dan menampilkan fungsionalitas penambahan angka yang persis sama seperti sebelum dilakukan hoisting.
- Proses penambahan nilai (**number++**) saat ini berada di dalam fungsi **onCreate()** pada deklarasi parameter lambda **onButtonClick**.
- State nilai angka (**number**) dideklarasikan di dalam fungsi **onCreate()**. Sementara nilai label tombol didefinisikan langsung sebagai argumen literal string **"Increment"** saat fungsi **CounterScreen** dipanggil di **onCreate()**.

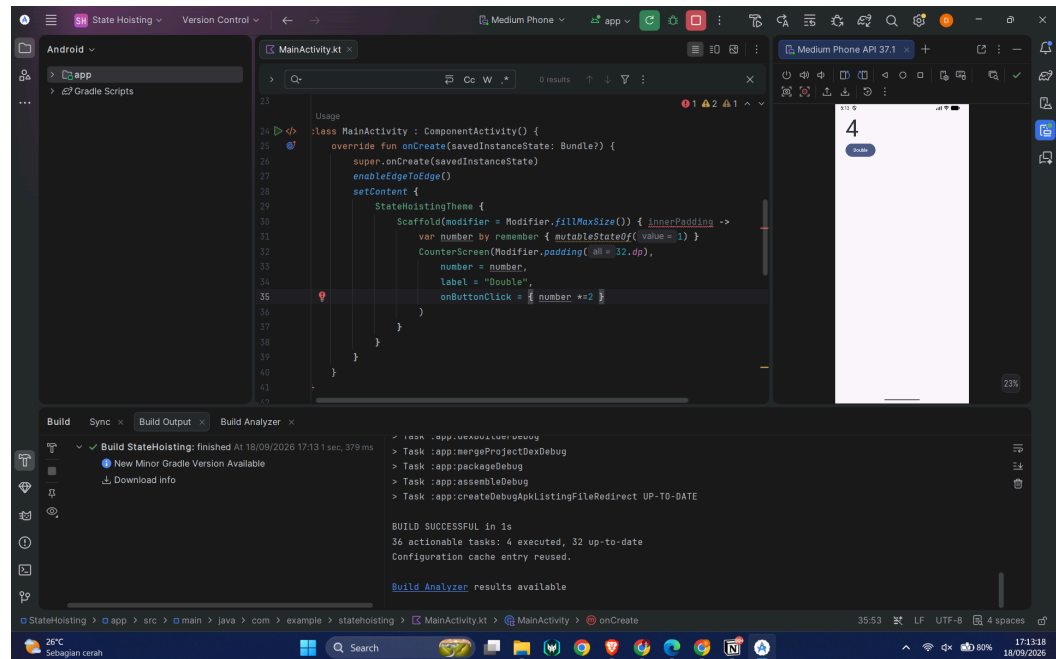


4. Pertanyaan:

- Bukti pengujian nilai variabel sebelum dan sesudah digandakan.
- Di bagian manakah dari kode program seluruh perubahan logika fungsional tersebut dilakukan?
- Apakah ada perubahan yang dilakukan di dalam fungsi Composable **CounterScreen** itu sendiri?

Jawaban:

- Diperlihatkan dengan tangkapan layar tampilan awal bernilai 1 dengan label "Double", lalu setelah diklik nilainya berubah menjadi 2 (atau kelipatannya).



- b. Seluruh perubahan logika dilakukan di dalam fungsi pemanggilnya, yaitu fungsi `onCreate()` pada [MainActivity.kt](#)
- c. Tidak ada perubahan sama sekali di dalam fungsi `CounterScreen`. Hal ini karena fungsi tersebut telah berstatus *stateless* dan hanya bertugas merender data yang diberikan.

5. Pertanyaan:

Apa arti dari hasil percobaan tersebut terkait dengan manfaat dari melakukan *state hoisting* dalam menghadapi perubahan kebutuhan fungsional aplikasi?

Jawaban:

State hoisting mewujudkan prinsip *Separation of Concerns* (pemisahan logika antarmuka dan logika bisnis) serta membuat komponen UI menjadi *reusable* dan *stateless*. Ketika ada kebutuhan bisnis baru (misalnya aturan perhitungan berubah), pengembang cukup mengubah logika di pemanggil tanpa perlu menyentuh atau merombak komponen UI `CounterScreen`.

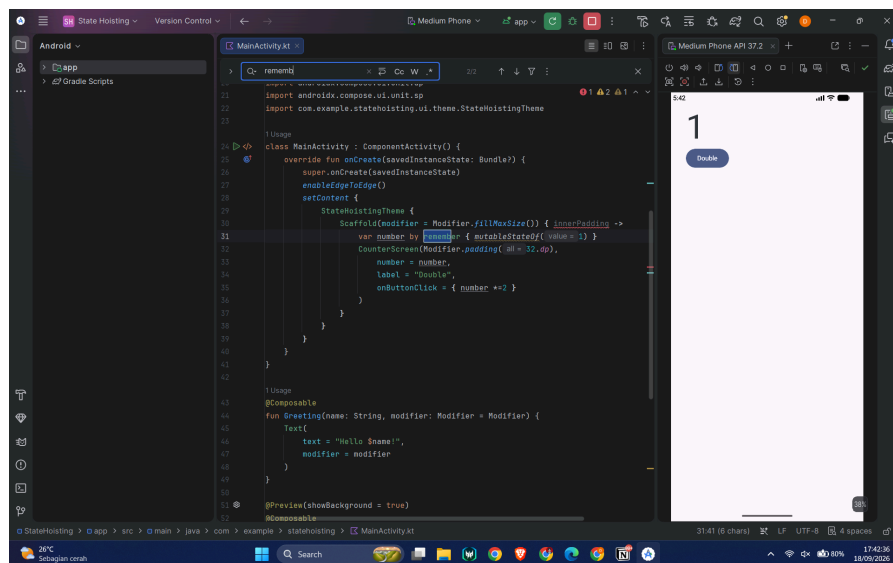
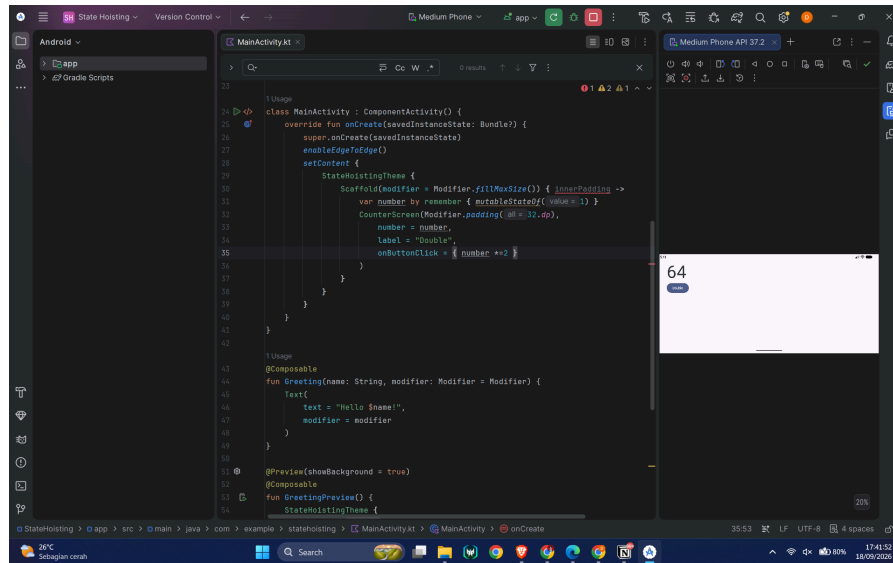
B. Perubahan Konfigurasi

1. Pertanyaan:

Berapa angka yang dicatat setelah tombol "Double" diklik beberapa kali pada posisi portrait?

Jawaban:

Angka berubah dari nilai awal 1 menjadi angka kelipatannya



2. Pertanyaan:

- Berapa angka yang ditampilkan setelah perangkat diputar ke posisi landscape (saat masih menggunakan **remember**)?
- Mengapa terjadi demikian?

Jawaban:

- a. Angka kembali ke nilai default awal, yaitu angka 1.

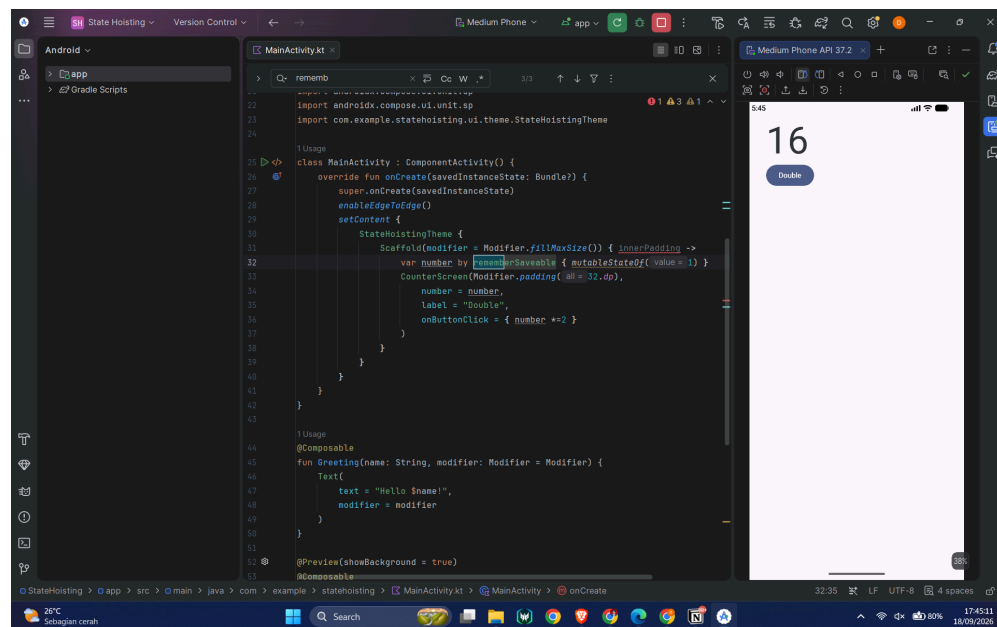
- b. Hal tersebut terjadi karena perubahan orientasi perangkat memicu *configuration change* yang menghancurkan dan membuat ulang *Activity* (*Activity recreation*). Konstruk `remember` hanya bertugas menjaga state selama proses rekomposisi, sehingga nilainya hilang saat *Activity* di-destroy.

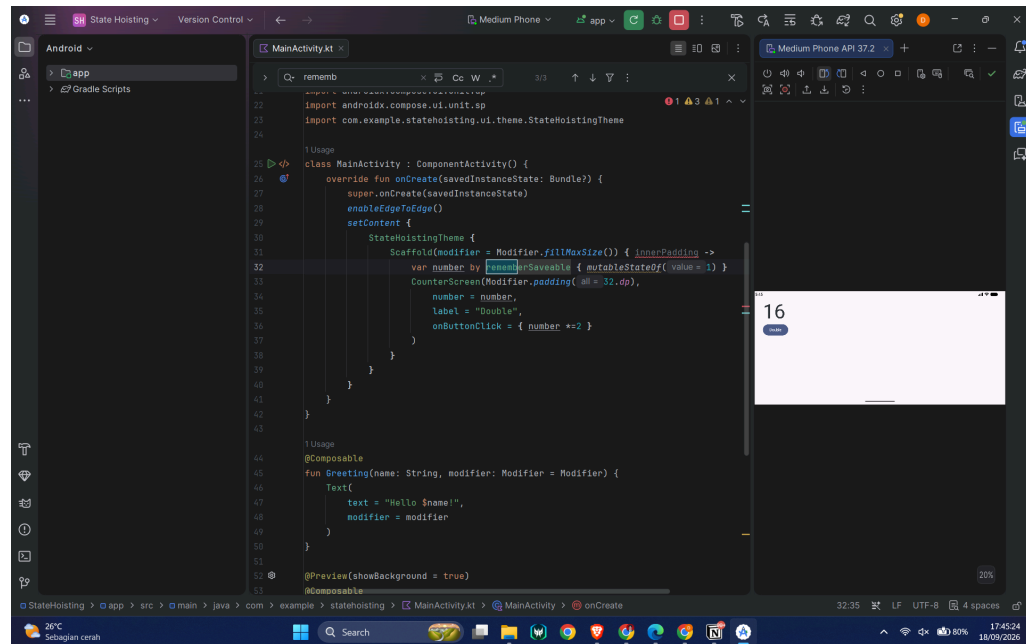
3. Pertanyaan:

- a. Berapa angka yang ditampilkan setelah perangkat diputar ke landscape jika deklarasi diganti menjadi `rememberSaveable`?
- b. Mengapa terjadi demikian?
- c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?

Jawaban:

- a. Angka yang ditampilkan tetap mempertahankan nilai terakhir sebelum layar diputar





- b. Karena **rememberSaveable** secara otomatis menyimpan data state ke dalam mekanisme bundle status instans (*Saved Instance State*), sehingga data tidak terhapus meskipun *Activity* dihancurkan dan dibuat ulang oleh sistem operasi.
- c. Konstruk **remember** hanya mempertahankan state melintasi rekomposisi antarmuka. Sedangkan **rememberSaveable** mampu mempertahankan nilai state melintasi rekomposisi sekaligus perubahan konfigurasi perangkat (seperti rotasi layar) maupun saat proses aplikasi dimatikan sementara oleh sistem Android.

4. Pertanyaan:

Beri contoh kasus nyata lain di mana penggunaan konstruk **rememberSaveable** tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Jawaban:

Contoh kasus nyata penerapannya adalah pada input formulir pendaftaran (teks pada **TextField**), status centang pada **Checkbox**, atau posisi penjelajahan (*scroll position*). Manfaatnya adalah menjaga kenyamanan pengguna (*user experience*) karena input data yang belum disimpan tidak akan hilang saat perangkat diputar secara tidak sengaja.

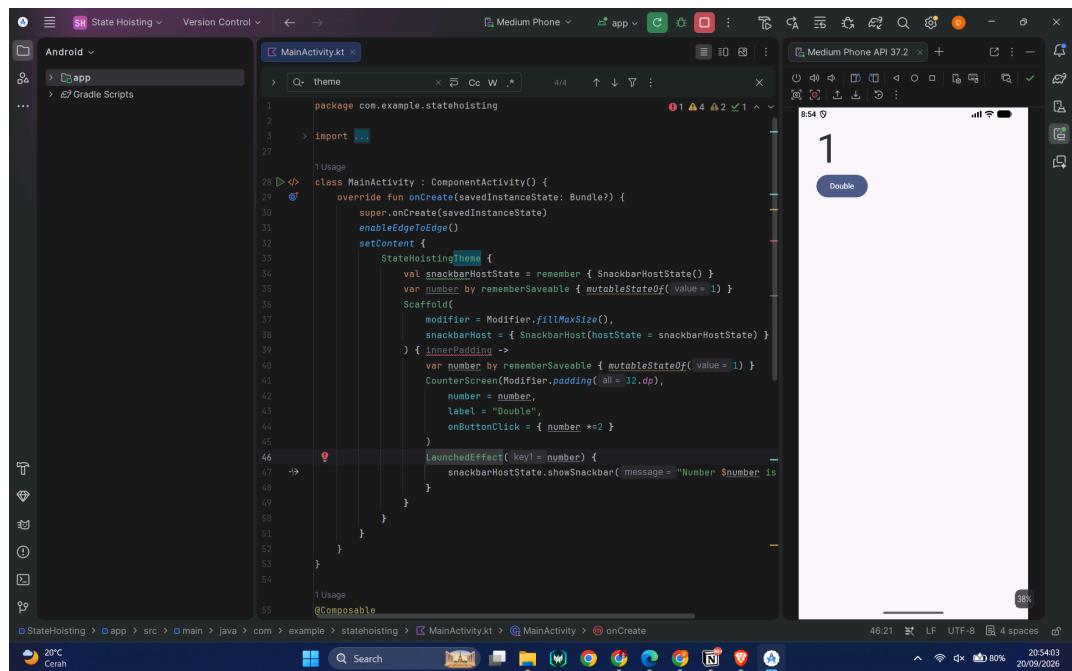
C. LaunchedEffect

1. Pertanyaan:

- Pada saat pertama kali program dijalankan tanpa mengklik tombol "Double", apa yang ditampilkan di dalam layar?
- Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?
- Apa yang memicu pesan pada **Snackbar** tersebut ditampilkan dan pada bagian kode program yang mana?

Jawaban:

- Muncul komponen **Snackbar** di bagian bawah layar yang menampilkan teks "Number 1 is shown!"



- LaunchedEffect** dirancang untuk langsung mengeksekusi blok coroutine di dalamnya saat composable terkait pertama kali memasuki pohon komposisi (*initial composition*).
- Pesan dipicu oleh pemanggilan fungsi penampil *side-effect* pada baris:

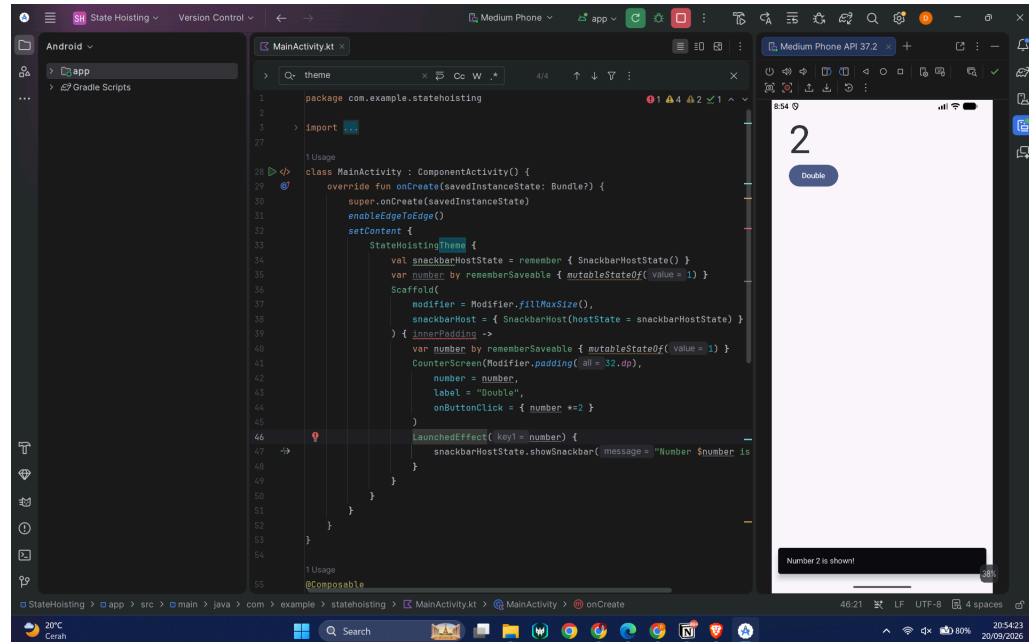
```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number is shown!")
}
```

2. Pertanyaan:

- Apa yang terjadi saat tombol "Double" diklik satu kali?
- Mengapa hal tersebut terjadi?

Jawaban:

- a. Angka di layar berlipat ganda menjadi 2 dan muncul **Snackbar** baru yang menampilkan pesan "Number 2 is shown!".



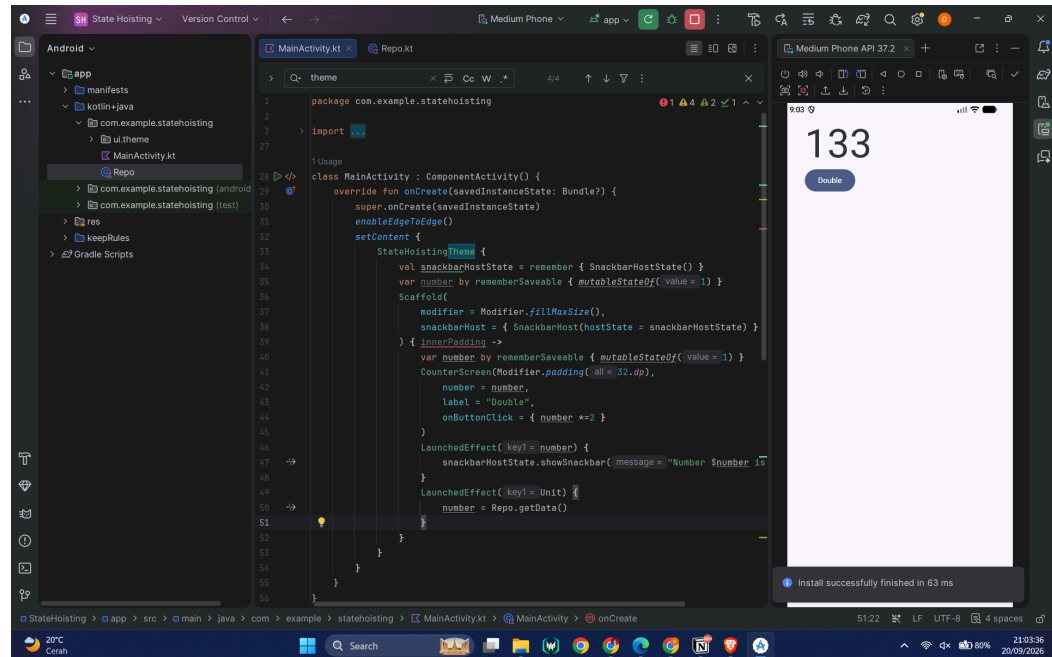
- b. Variabel **number** dipasang sebagai parameter kunci (*key*) dari **LaunchedEffect(number)**. Saat tombol ditekan, nilai **number** berubah, sehingga coroutine yang lama dibatalkan dan coroutine baru dijalankan ulang dengan nilai *key* yang baru.

3. Pertanyaan:

- a. Pada pra-percobaan 2, apa yang ditampilkan pada layar pada saat pertama kali program dijalankan?
- b. Apa yang tampil di layar setelah menunggu selama dua detik?
- c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada **Snackbar** tersebut dan pada bagian kode program yang mana?

Jawaban:

- a. Layar menampilkan angka 1 dan pesan **Snackbar** "Number 1 is shown!".
- b. Setelah jeda 2 detik, angka pada layar otomatis berganti menjadi bilangan acak (antara 100 hingga 1000) dan **Snackbar** baru muncul menampilkan pesan nilai acak tersebut.



- c. Perubahan dipicu oleh blok `LaunchedEffect(Unit)` yang menjalankan fungsi coroutine `Repo.getData()`. Begitu nilai acak diperoleh setelah delay 2 detik, baris `number = Repo.getData()` memperbarui state `number`, yang kemudian secara otomatis memicu pemanggilan `LaunchedEffect(number)` untuk menampilkan pesan baru di `Snackbar`.

4. Pertanyaan:

- a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`!
- b. Kapan operasi di dalam `LaunchedEffect(state)` dan `LaunchedEffect(Unit)` dijalankan?

Jawaban:

- a. `LaunchedEffect(Unit)` menggunakan key konstan yang tidak pernah berganti, sehingga hanya dijalankan tepat satu kali selama daur hidup composable berada dalam komposisi. Sedangkan `LaunchedEffect(state)` memiliki key dinamis yang akan me-restart blok coroutine setiap kali nilai state yang dipantau mengalami perubahan.
- b. Operasi di dalam `LaunchedEffect(Unit)` dieksekusi hanya pada saat *initial composition* (saat layar pertama dibuka). Operasi di dalam `LaunchedEffect(state)` dieksekusi pada saat *initial composition* dan akan dieksekusi ulang secara otomatis setiap kali parameter nilai state yang diumpankan berubah.